🔷 1. INTRODUCTION

What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines:

- **Information Retrieval (Vector Database)**
- **Text Generation (LLM)**

Instead of relying only on the model's knowledge, RAG retrieves relevant information from external data sources (like PDFs) and uses it to generate accurate answers.

Why is RAG Needed?

Traditional chatbots:

- Hallucinate (give wrong answers)
- Lack real-time knowledge
- Cannot use custom data

RAG solves this by:

- Using real documents
- Reducing hallucination
- Improving accuracy

Use Case: Customer Support Assistant

This system:

- Answers user queries from company documents
- Handles unknown queries via human escalation
- Mimics real-world support systems

🔷 2. HIGH-LEVEL DESIGN (HLD)

2.1 System Overview

Problem Definition

Customer support systems struggle with:

- High query volume
- Repetitive questions
- Dependency on human agents

Scope

- Input: User query
- Source: PDF knowledge base
- Output:
    - AI-generated answer
    - OR Human-assisted response

2.2 Architecture Overview

User (CLI/Web)

↓

LangGraph Workflow

↓

Processing Node

- Retrieval (ChromaDB)

- LLM (Ollama)

↓

Router

↓ ↓

Output HITL

↓ ↓

Answer Human Agent

2.3 Component Description

1. Document Loader

- Loads PDF using PyPDFLoader
- Extracts raw text

2. Chunking Strategy

- Chunk size: 800 tokens
- Overlap: 100 tokens
- Maintains context continuity

3. Embedding Model

- Model: `nomic-embed-text`
- Converts text → vector embeddings

4. Vector Store

- ChromaDB
- Stores embeddings locally

5. Retriever

- Retrieves top-k similar chunks
- Uses cosine similarity

6. LLM

- Model: `phi` (Ollama)
- Generates answers using context

7. Graph Workflow Engine

- LangGraph
- Controls execution flow

8. Routing Layer

- Decides:
  - Answer → Output
  - Low confidence → HITL

9. HITL Module

- Human intervention system
- Handles complex queries

2.4 Data Flow

Document Pipeline

PDF → Text → Chunks → Embeddings → ChromaDB

Query Lifecycle

1. User inputs query
2. Retriever fetches context
3. LLM generates answer
4. Router evaluates confidence

5. Output OR escalation

2.5 Technology Choices

Why ChromaDB

- Lightweight
- Fast
- Local storage

Why LangGraph

- Graph-based execution
- Supports branching
- Maintains state

LLM Choice

- Ollama (phi model)
- Free and local

Other Tools

- LangChain
- Python

2.6 Scalability Considerations

Large Documents

- Chunking strategy
- Efficient indexing

High Query Load

- Caching
- Parallel retrieval

Latency

- Reduce top-k
- Optimize embeddings

🔷 3. LOW-LEVEL DESIGN (LLD)

3.1 Module-Level Design

Document Processing Module

- Loads and parses PDF

Chunking Module

- Splits text into chunks

Embedding Module

- Converts text into vectors

Vector Storage Module

- Stores embeddings in ChromaDB

Retrieval Module

- Fetches relevant chunks

Query Processing Module

- Generates answer

Graph Execution Module

- Executes workflow

HITL Module

- Handles escalation

3.2 Data Structures

Document

```
{
 "id": "doc1",
"text": "content"
}
```

Chunk

```
{
  "chunk_id": "c1",
  "text": "...",
```

```
  "embedding": [0.12, 0.98]
}
```

Query-Response

```
{
  "query": "...",
  "answer": "...",
  "confidence": 0.8
}
```

Graph State

```
{
  "query": "",
  "context": "",
  "answer": "",
  "confidence": 0
}
```

3.3 Workflow Design (LangGraph)

Nodes

- Processing Node
- Output Node
- HITL Node

Flow

```
Input → Process → Router → Output / HITL
```

3.4 Conditional Routing Logic

Answer if:

- Confidence ≥ threshold
- Context available

Escalate if:

- Low confidence
- Missing context
- Complex query

3.5 HITL Design

Trigger:

- Low confidence OR unknown answer

Flow:

1. System detects uncertainty
2. Escalates to human
3. Human provides answer
4. System returns response

3.6 API / Interface Design

Input

```
{
  "query": "What is refund policy?"
}
```

Output

```
{
  "answer": "...",
  "confidence": 0.85,
  "source": "RAG/HITL"
}
```

3.7 Error Handling

Issue	Solution
No data	Escalate
LLM failure	Retry
Invalid input	Validate

🔷 4. TECHNICAL DOCUMENTATION

4.1 Architecture Explanation

- Documents stored as embeddings
- Queries retrieve context
- LLM generates answers
- LangGraph controls workflow

4.2 Design Decisions

Chunk Size

- 800 tokens → balance

Embedding Strategy

- Semantic embeddings

Retrieval

- Top-k = 3

Prompt Design

- Context-based answering
- “I don’t know” fallback

4.3 Workflow Explanation

- State flows between nodes
- Conditional routing applied
- HITL integrated

4.4 Conditional Logic

- Based on confidence
- Based on context availability

4.5 HITL Implementation

Benefits:

- Improves reliability
- Handles edge cases

Limitations:

- Slower response
- Requires human input

4.6 Challenges & Trade-offs

Trade-off	Explanation
Accuracy vs Speed	More retrieval = slower
Cost vs Performance	Better models cost more
Chunk size vs Context	Larger chunks = better context

4.7 Testing Strategy

Sample Queries:

- Refund policy
- Delivery info
- Unknown queries

Metrics:

- Accuracy
- Response time

4.8 Future Enhancements

- Multi-document support
- Memory integration
- Feedback loop
- Cloud deployment

🔷 5. CONCLUSION

This project demonstrates:

- Real-world RAG implementation
- Workflow-based AI system
- Decision-making capability

👉 It is not just a chatbot, but a **scalable AI system**